

КУРС

Базы данных

PostgreSQL

Реляционная модель: ключи, связи и нормальные формы

Таблица - это не Excel

Таблица PostgreSQL

строка = один объект

столбец = одно свойство

тип столбца заранее понятен

порядок задаём ORDER BY

Excel

ячейки можно смешивать

порядок виден глазами

формат часто важнее структуры

Таблица похожа на лист, но живет по правилам данных

Сущность, экземпляр и атрибут

Сущность = таблица

тип объектов

product

customer

orders

Экземпляр = строка

конкретный объект

товар 101

клиент 7

заказ 501

Атрибут = столбец

свойство объекта

name

price

email

Зачем строке нужен ключ

product_id	name	price
101	Кабель USB-C	990.00
102	Кабель USB-C	1490.00

Как отличить?

name

повторяется

price

может измениться

product_id

однозначный и стабильный

Ключ отвечает: «какая именно это строка?»

Естественный и суррогатный ключ

Естественный ключ

существует в предметной области

sku = NB-15-001

email - иногда кандидат

МОЖЕТ ИЗМЕНИТЬСЯ

Суррогатный ключ

создаем специально

product_id = 101

customer_id = 7

стабилен для связей

PRIMARY KEY выбираем один другие уникальные поля -> UNIQUE

Кандидатный ключ - все варианты которые подходят на роль уникального идентификатора, а РК это один из кандидатных ключей, который мы выбрали

PRIMARY KEY: «кто это?»

```
shop=# INSERT INTO product(product_id,name,price,stock)
shop=# VALUES (101,'Кабель USB-C',990.00,20);
INSERT 0 1
```

```
shop=# INSERT INTO product(product_id,name,price,stock)
shop=# VALUES (101,'Другой кабель',1290.00,5);
ERROR: duplicate key value violates unique constraint
        "product_pkey"
DETAIL: Key (product_id)=(101) already exists.
```

PRIMARY KEY

уникален

не NULL

одна строка = один id

ошибка защищает данные

FOREIGN KEY: «на кого ссылаемся?»

```
shop=# INSERT INTO customer(customer_id,full_name)
shop=# VALUES (7,'Анна Иванова');
INSERT 0 1
```

```
shop=# INSERT INTO orders(order_id,customer_id,status)
shop=# VALUES (501,7,'NEW');
INSERT 0 1
```

```
shop=# INSERT INTO orders(order_id,customer_id,status)
shop=# VALUES (502,999,'NEW');
ERROR: insert or update on table "orders" violates
foreign key constraint "orders_customer_id_fkey"
```

FOREIGN KEY

orders.customer_id

--->

customer.customer_id

ссылка не может вести в
пустоту

Связь 1:N: клиент и заказы

customer

7 Анна Иванова

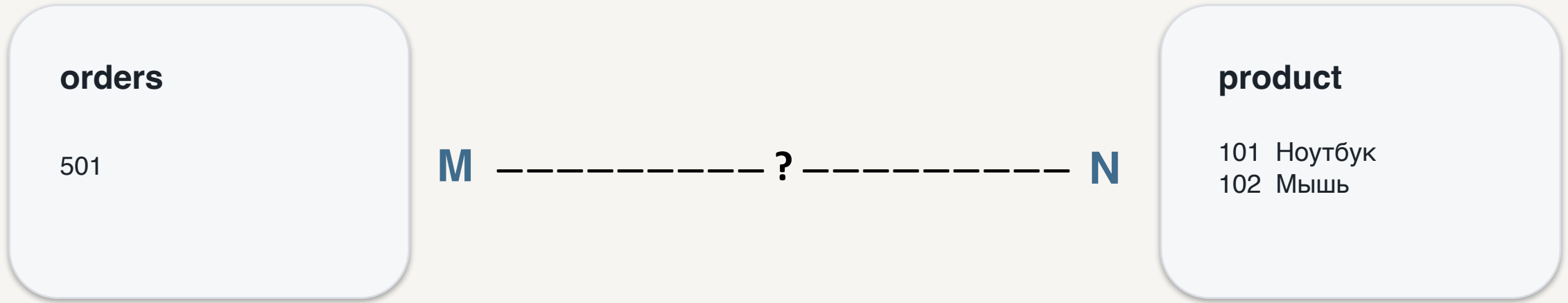
1 — — > N

orders

order_id	customer_id	status
501	7	NEW
508	7	PAID
520	7	DONE

Внешний ключ находится на стороне «много»

Связь M:N: заказ и товары



Плохо:
products = '101,102,103'

Решение → отдельная таблица
order_item

order_item: связь с собственными данными

order_id	product_id	quantity	unit_price
501	101	1	74990.00
501	102	2	1490.00

**Что принадлежит
позиции заказа?**

quantity
unit_price
discount

Цена каталога может измениться, а вот цена покупки - это исторический факт

Почему данные начинают ломаться

order	customer	phone	product	product_name
501	Анна	+7 900...	101	Ноутбук
508	Анна	+7 900...	102	Мышь
520	Анна	+7 900...	101	Ноутбук

Обновление

телефон исправили не везде

Удаление

потеряли нужный факт

Вставка

товар зависит от заказа

Нормализация = каждый факт хранится в подходящем месте

1НФ: одно поле - одно значение

Плохо

orders.products

'Ноутбук, Мышь, Кабель'

одна ячейка = список

1НФ

orders

один заказ = одна строка

order_item

один товар заказа = одна строка

Отношение находится в 1НФ, если каждый его атрибут содержит только атомарное значение, а повторяющиеся группы атрибутов отсутствуют

1НФ: не прячем повторяющиеся значения внутри одной ячейки

2НФ: зависит от всего ключа

order_id	product_id	quantity	product_name
501	101	1	Ноутбук
508	101	2	Ноутбук
520	101	1	Ноутбук

Ключ

(order_id, product_id)

quantity

зависит от всей пары

product_name

зависит только от product_id

2НФ: факт о товаре переносим в product

Отношение находится во 2НФ, если оно находится в 1НФ, и каждый не ключевой атрибут полностью функционально зависит от каждого кандидатского ключа, а не от его части

3НФ: не храним цепочку зависимостей

product_id	name	category_id	category_name
101	Ноутбук	10	Ноутбуки
104	Ультрабук	10	Ноутбуки
205	Мышь	20	Аксессуары

Зависимость

product_id



category_id

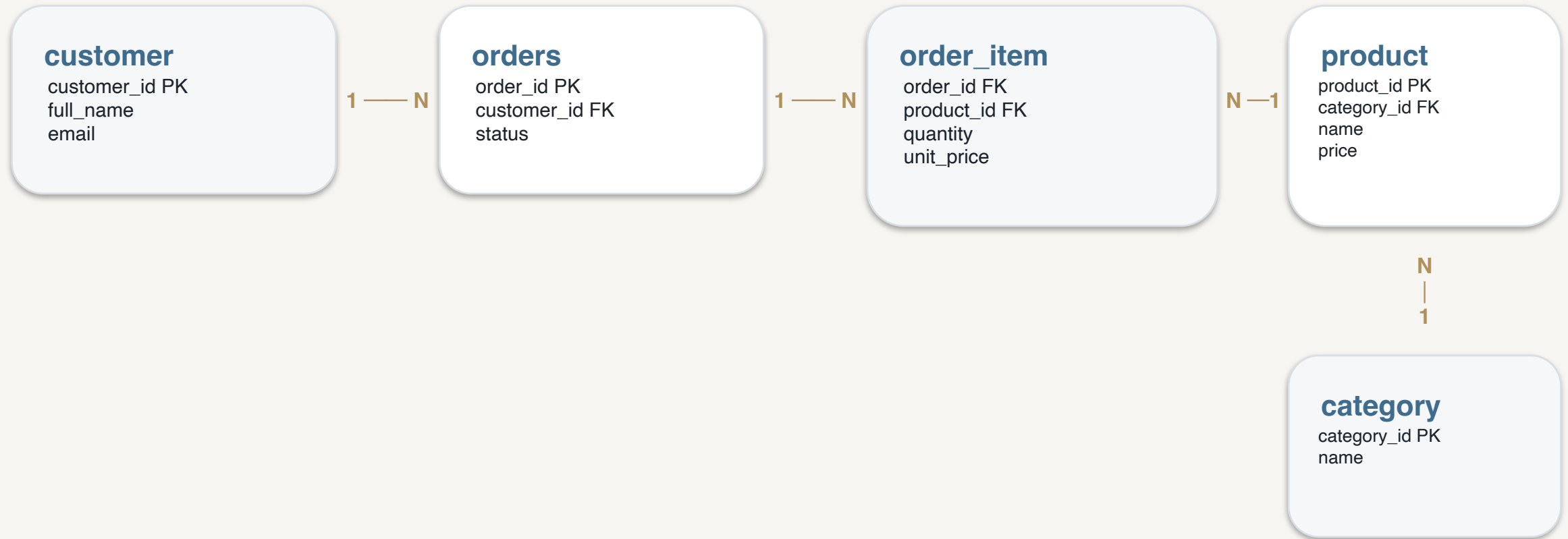


category_name

3НФ: название категории хранится в category

Отношение находится в 3НФ, если оно находится во 2НФ, и не ключевые атрибуты не имеют транзитивных функциональных зависимостей от кандидатных ключей через другие атрибуты

Магазин после нормализации



Один факт - одно естественное место хранения

Проверяем ключи и ссылки в PostgreSQL

```
shop=# INSERT INTO customer(customer_id,full_name)
shop=# VALUES (7,'Анна Иванова');
INSERT 0 1
```

```
shop=# INSERT INTO orders(order_id,customer_id,status)
shop=# VALUES (501,7,'NEW');
INSERT 0 1
```

```
shop=# INSERT INTO orders(order_id,customer_id,status)
shop=# VALUES (502,999,'NEW');
ERROR: violates foreign key constraint
```

Что проверила база?

PRIMARY KEY

не дает повторить id

FOREIGN KEY

не дает ссылаться на 999

ограничения = исполняемые
правила

Читаем заказ как связанную модель

```
shop=# SELECT o.order_id, c.full_name,  
shop=#       p.name, oi.quantity, oi.unit_price  
shop=# FROM orders o  
shop=# JOIN customer c ON c.customer_id = o.customer_id  
shop=# JOIN order_item oi ON oi.order_id = o.order_id  
shop=# JOIN product p ON p.product_id = oi.product_id  
shop=# WHERE o.order_id = 501;
```

order_id	full_name	name	quantity	unit_price
501	Анна Иванова	Ноутбук	1	74990.00
501	Анна Иванова	Мышь	2	1490.00

(2 rows)

**Храним факты раздельно, а
нужный вид собирает
запрос**

В запросе таблица orders o - где o это альяс таблицы, то есть временное альтернативное имя таблицы в рамках SQL-запроса

Какие базы данных бывают кроме реляционных

Модель хранения выбирают под задачу. «Нереляционная» не значит «лучше» - просто данные устроены иначе.

Документные

карточка товара целиком

JSON-документ

MongoDB

товар + характеристики + фото

Ключ → значение

корзина покупателя

user:101 → корзина

Redis

быстрый доступ по известному ключу

Графовые

рекомендации и связи

клиент → купил → товар

Neo4j

важны цепочки связей

Wide-column

клики и события магазина

много записей, много серверов

Cassandra

огромные потоки однотипных данных

А PostgreSQL?

Реляционная СУБД: таблицы + связи + SQL. При этом PostgreSQL умеет хранить и JSONB

Для нашего магазина основная модель — реляционная: customer → orders → order_item → product

Цена товара: NUMERIC, FLOAT, INTEGER или BIGINT?

Одна и та же цена 79 990,50 ₽ - четыре типа данных ведут себя по-разному

Тип	Что получится	Что важно
<code>numeric(10,2)</code>	79 990.50	точно хранит копейки для денег
<code>float / double precision</code>	79 990.5	приближенное двоичное число измерения, расчёты
<code>integer</code>	79 991	копейки потеряны: округление 4 байта, целые числа
<code>bigint</code>	79 991	то же округление, но диапазон больше 8 байт , очень большие целые

```
shop=# SELECT
shop-# 79990.50::numeric(10,2) AS
numeric_price,
shop-# 79990.50::float AS float_price,
shop-# 79990.50::integer AS int_price,
shop-# 79990.50::bigint AS bigint_price;

 numeric_price | float_price | int_price |
-----+-----+-----+----
79990.50      | 79990.5     | 79991     |
79991
(1 row)
```

Цена магазина → NUMERIC FLOAT — приблизительный INT/BIGINT — только целые

Важно: при приведении 79990.50 к INTEGER/BIGINT PostgreSQL округлит до 79991

КУРС

Базы данных

PostgreSQL

Проектирование базы данных и ER- модель

Проектирование начинается не с CREATE TABLE

Слишком рано

CREATE TABLE ...

varchar(100)?

NULL или NOT NULL?

а что мы вообще храним?

Сначала смысл

что существует?

какие свойства?

как связано?

что обязательно?

Требования → модель → таблицы → SQL

Шаг 1. Выписываем факты из задачи

«Клиент делает заказы. Заказ содержит товары. Товар относится к категории. Для товара в заказе сохраняются количество и цена покупки»

Существительные

клиент заказ товар категория

Глаголы

делает содержит относится

Характеристики

количество цена покупки

Сначала выписываем факты. Типы данных - потом

Шаг 2. Находим сущности

customer

живёт независимо
есть свой id
делает заказы

product

есть в каталоге
есть свой id
имеет цену

orders

самостоятельный факт
есть свой id
имеет статус

category - тоже сущность: ее используют многие товары

Шаг 3. Выбираем атрибуты

сущность

атрибуты

customer

customer_id full_name email

product

product_id name price stock category_id

orders

order_id customer_id created_dt status

**Один атрибут =
один понятный
факт**

Плохо:
customer_info =
'Анна, +7999, Москва'

Детализация зависит от задач системы

Атрибут или отдельная сущность?

Атрибут

phone

одно значение

нет своих свойств

нужен только как контакт

Отдельная сущность

customer_phone

телефонов много

type · verified · created_dt

есть собственные правила

Шаг 4. Выбираем ключи

объект	ключ	почему
customer	customer_id	email может измениться
product	product_id	name не уникален
orders	order_id	нужна стабильная ссылка
category	category_id	name можно переименовать

order_item

вариант 1

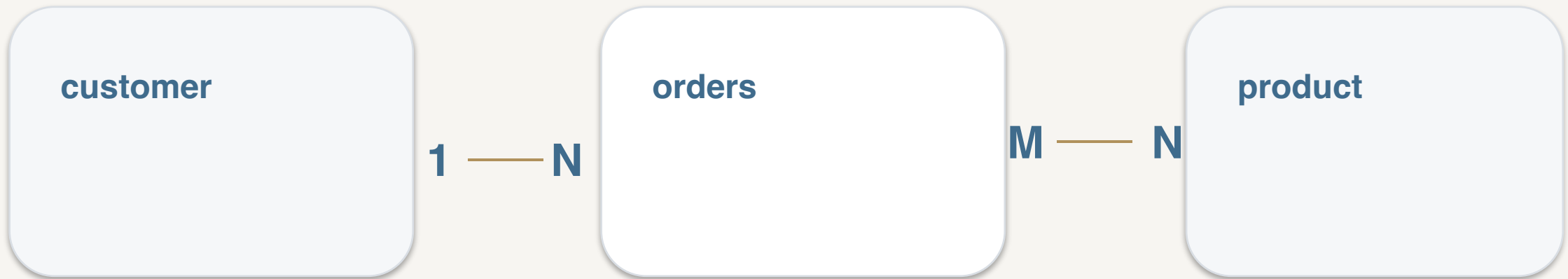
(order_id, product_id)

вариант 2

order_item_id

Ключ фиксирует правило уникальности

Шаг 5. Рисуем связи и кардинальность



Заказ ↔ товар = M:N

Кардинальность отвечает на вопрос: «сколько объектов с каждой стороны?»